

Lekce 10: Strukturování testovacích sad a testovacích případů

Cíle

- Organizovat testy podle osvědčených postupů pro udržovatelnost a škálovatelnost.
- Využívat příkazy Cypress pro tvorbu čistého, znovupoužitelného testovacího kódu.
- Implementovat správné procedury pro přípravu a úklid testů (setup a teardown).
- Používat štítky (tags) a pojmenovací konvence pro kategorizaci testů pro selektivní spouštění.

Obsah lekce

A. Organizace testů

- **Používání bloků `describe()` a `it()` :**

- **Blok `describe()` :**

Slouží ke sdružení souvisejících testů do jedné testovací sady. Pomáhá logicky organizovat testy podle funkce nebo vlastnosti.

- **Příklad:**

```
describe('Přihlašovací funkce', () => {  
  // Sem patří testovací případy týkající se přihlašování.  
});
```

- **Blok `it()` :**

Definuje jednotlivý testovací případ, který ověřuje konkrétní chování.

- **Příklad:**

```
it('by se měl úspěšně přihlásit s platnými údaji', () => {  
  // Kroky testu a aserce.  
});
```

- **Seskupování souvisejících testů:**

- Organizujte testy podle uživatelských cest (např. přihlášení, registrace, dashboard) nebo podle funkcí.
 - Používejte vnořené bloky `describe()` , pokud je potřeba vytvořit podskupiny pro jemnější organizaci.

Příklad vnořených bloků `describe()`

Vnořené bloky `describe` pomáhají dále seskupovat a organizovat testy, zejména při práci s komplexními uživatelskými scénáři nebo různými aspekty téže funkce.

Příklad:

```
describe('Uživatelský účet', () => {
  describe('Přihlašovací funkce', () => {
    it('by se měl úspěšně přihlásit s platnými údaji', () => {
      // Logika testu přihlášení
    });

    it('by měl zobrazit chybu při neplatných údajích', () => {
      // Logika testu pro chybový případ
    });
  });

  describe('Registrační funkce', () => {
    it('by měl zaregistrovat nového uživatele s platnými údaji', () => {
      // Logika testu registrace
    });

    it('by měl zobrazit chyby při chybějících polích', () => {
      // Logika testu pro validační chyby
    });
  });
});
```

Správné pojmenování bloků describe() a it()

Pojmenovací konvence by měly být jasné, popisné a měly by následovat konzistentní vzor. Dobrá praxe je používat přirozený jazyk k popsání toho, co se testuje a jaký je očekávaný výsledek.

- **Názvy bloků describe():**

Používejte podstatné jméno nebo název vlastnosti, který jasně identifikuje předmět testů.

Příklady:

- describe('Přihlašovací funkce', () => { ... })
- describe('Registrace uživatele', () => { ... })
- describe('Operace s nákupním košíkem', () => { ... })

- **Názvy bloků it():**

Používejte větu nebo frázi, která deklaruje očekávané chování. Tam, kde je to možné, v testovacích názvech použijte vzor AAA (Arrange-Act-Assert).

Příklady:

- it('by měl zobrazit uvítací zprávu při úspěšném přihlášení', () => { ... })
- it('by měl zobrazit chybu, když během registrace chybí povinná pole', () => { ... })
- it('by měl přidat položku do košíku a odpovídajícím způsobem upravit celkovou cenu', () => { ... })

Používání AAA (Arrange-Act-Assert) v testech a jejich názvech

Vzor AAA:

Tento vzor (Arrange-Act-Assert) strukturuje testy do tří jasných fází:

- **Arrange (Příprav):** Nachystej testovací scénář, inicializuj data a připrav prostředí.

- **Act (Jednej):** Proved' akci, kterou chceš testovat.
- **Assert (Ověř):** Ověř, že výsledek odpovídá očekávání.

V názvech testů:

Zahrnutí náznaků vzoru AAA do názvů testů může objasnit záměr:

- **Příklad názvu testu:**

```
it('připraví platná přihlašovací data, odešle formulář a ověří zobrazení uvítací zprávy', () => { ... })
```

Příklad testu podle AAA:

```
it('by se měl úspěšně přihlásit s platnými údaji', () => {
  // Arrange
  const validUsername = 'demoUser';
  const validPassword = 'demoPass';

  // Act
  cy.get('[data-testid="username-input"]').type(validUsername);
  cy.get('[data-testid="password-input"]').type(validPassword);
  cy.get('[data-testid="login-button"]').click();

  // Assert
  cy.get('[data-testid="login-success-message"]').should('be.visible');
});
```

B. Metody přípravy a úklidu (Setup a Teardown)

- **Přehled hooků:**

- **Hook before() :**
Spouští se jednou před všemi testy v bloku `describe()` .
- **Hook beforeEach() :**
Spouští se před každým jednotlivým testem, užitečné pro reset stavu nebo navigaci na společný výchozí bod.
- **Hook after() :**
Spouští se jednou po všech testech v bloku `describe()` , často se využívá pro úklid.
- **Hook afterEach() :**
Spouští se po každém testu, užitečné pro resetování změn vzniklých během testu.

- **Praktické využití:**

- **Předpoklady:**
Příprava testovacích dat nebo přihlášení uživatele před spuštěním testů.
- **Úklid:**
Resetování dat nebo mazání dat ze session storage, aby testy byly izolované.
- **Příklad:**

```

describe('Přihlášení uživatele', () => {
  before(() => {
    // Spustí se jednou před všemi testy
    cy.log('Spouštím sadu testů pro přihlášení');
  });

  beforeEach(() => {
    // Přejdi na stránku přihlášení před každým testem
    cy.visit('/login');
  });

  afterEach(() => {
    // Vymaž cookies nebo local storage dle potřeby
    cy.clearCookies();
  });

  after(() => {
    cy.log('Testovací sada pro přihlášení dokončena');
  });

  it('by se měl úspěšně přihlásit s platnými údaji', () => {
    // Testovací kód
  });
});

```

C. Tagování a kategorizace testů

- **Používání pojmenovacích konvencí:**

- Předpona nebo přípona v názvu testu indikuje jeho kategorii (např. @smoke , @regression , @login).
- **Příklad:**

```

describe('Přihlášení uživatele @smoke', () => { ... });

```

- **Pluginy pro tagování v Cypress:**

- **Cypress-Grep:**

Tento plugin umožňuje filtrovat testy podle štítků při jejich spouštění.

- **Instalace:**

```

npm install cypress-grep --save-dev

```

- **Použití:**

Spuštění testů s konkrétním tagem:

```

npx cypress run --env grep=@login

```

- **Vlastní tagování:**

Pokud je potřeba další organizace, použijte komentáře nebo vlastní metadata.

D. Funkcionalita vlastních Cypress příkazů

- **Vestavěné příkazy Cypress:**

- Příkazy jako `cy.visit()`, `cy.get()`, `cy.click()`, `cy.type()` a další slouží pro interakci s aplikací.

- **Příklad:**

```
cy.visit('/login');
cy.get('[data-testid="username-input"]').type('demoUser');
cy.get('[data-testid="password-input"]').type('demoPass');
cy.get('[data-testid="login-button"]').click();
```

- **Vlastní příkazy:**

Vlastní příkazy v Cypress

Co jsou vlastní příkazy?

Vlastní příkazy v Cypress jsou funkce, které si definujete (obvykle v `cypress/support/commands.js`) a které zahrnují opakující se akce nebo složitější logiku. Pomáhají zredukovat duplicitní kód, zpřehlednit testy a zvyšují znovupoužitelnost.

Výhody:

- **Znovupoužitelnost:**

Stačí napsat příkaz jednou a použít ho v různých testech.

- **Čitelnost:**

Vlastní příkazy tvoří vyšší abstrakci a činí testy srozumitelnější.

- **Udržitelnost:**

Úpravy v běžné funkcionalitě stačí provést jen na jednom místě.

Nevýhody:

- **Režie abstrakce:**

Nadměrné používání nebo přílišná abstrakce vlastních příkazů může komplikovat debugování.

- **Potenciál skryté logiky:**

Pokud vlastní příkaz obsahuje příliš mnoho logiky, může být těžké pochopit, co se v testu skutečně děje.

Příklad definice vlastních příkazů:

V `cypress/support/commands.js`:

```
// Vlastní příkaz pro přihlášení
Cypress.Commands.add('login', (username, password) => {
  cy.get('[data-testid="username-input"]').clear().type(username);
  cy.get('[data-testid="password-input"]').clear().type(password);
  cy.get('[data-testid="login-button"]').click();
});

// Vlastní příkaz pro registraci nového uživatele
Cypress.Commands.add('registerUser', (user) => {
  cy.get('[data-testid="reg-username-input"]').clear().type(user.username);
  cy.get('[data-testid="reg-email-input"]').clear().type(user.email);
  cy.get('[data-testid="reg-password-input"]').clear().type(user.password);
  cy.get('[data-testid="reg-submit-button"]').click();
});
```

Použití v testech:

```
describe('Ověření uživatele', () => {
  it('by se měl úspěšně přihlásit', () => {
    // Použití vlastního příkazu login
    cy.login('demoUser', 'demoPass');
    cy.get('[data-testid="login-success-message"]').should('be.visible');
  });

  it('by měl zaregistrovat nového uživatele', () => {
    const user = {
      username: 'newUser',
      email: 'newuser@example.com',
      password: 'newPassword123'
    };

    // Použití vlastního příkazu pro registraci
    cy.registerUser(user);
    cy.get('[data-testid="reg-success-message"]').should('be.visible');
  });
});
```

Další výhody vlastních příkazů:

- **Zapouzdření:**
Zapouzdříte běžné uživatelské interakce a zredukujete opakující se kód.
- **Abstrakce:**
Abstrahujte nízkoúrovňové příkazy Cypress a zaměřte se na vyšší logiku testu.
- **Jednoduchá údržba:**
Pokud se funkčnost změní, stačí upravit vlastní příkaz místo všech testů, které stejnou akci využívají.
- Rozšiřte funkcionalitu Cypress přidáváním vlastních příkazů v souboru `cypress/support/commands.js`.
- **Příklad:**

```
// V cypress/support/commands.js
Cypress.Commands.add('login', (username, password) => {
  cy.get('[data-testid="username-input"]').type(username);
  cy.get('[data-testid="password-input"]').type(password);
  cy.get('[data-testid="login-button"]').click();
});
// Použití v testech:
cy.login('demoUser', 'demoPass');
```

- **Osvědčené postupy:**

- Udržujte příkazy popisné a snadno udržovatelné.
- Používejte vlastní příkazy pro omezení opakování a centralizaci logiky.
- Vždy zajistěte, aby účel každého příkazu byl srozumitelný i pro čtenáře testu.

E. Struktura testování Cypress projektů

Osvědčené postupy:

- **Struktura adresářů:**

- **cypress/fixtures:** Obsahuje externí datové soubory (JSON, atd.) pro testovací data.
- **cypress/e2e:** Obsahuje všechny testovací soubory rozdělené podle funkcí (např. login.spec.js, registration.spec.js).
- **cypress/plugins:** Obsahuje pluginy a modifikace výchozího chování Cypressu.
- **cypress/support:**
 - **commands.js:** Definice vlastních příkazů.
 - **index.js:** Globální konfigurace a kód pro nastavení Cypress testů.

- **Organizace testů:**

- Seskupujte testy pomocí bloků `describe()` podle funkcí nebo modulů.
- Používejte vnořené bloky `describe()` pro podfunkce.
- Konzistentně dodržujte pojmenovací konvenci souborů (např. login.spec.js pro testy související s přihlášením).

- **Konfigurace prostředí:**

- Pro správu nastavení jako baseUrl, timeouty a proměnné prostředí používejte konfigurační soubor (cypress.config.js).
- Odděluje prostředí-specifické testy pojmenováním nebo pluginy jako Cypress-grep.

- **Udržovatelnost a znovupoužitelnost:**

- Pište vlastní příkazy pro odstranění duplicitního kódu.
- Externí data ukládejte do fixtures, aby byly testy čisté a přehledné.
- Pro přehlednost testu dodržujte AAA vzor (Arrange-Act-Assert).

- **Příklad struktury adresářů:**

```
cypress/
├─ fixtures/
│   └─ users.json
├─ integration/
│   ├── login.spec.js
│   └─ registration.spec.js
├─ plugins/
│   └─ index.js
└─ support/
    ├── commands.js
    └─ index.js
```

Praktické aktivity

1. Strukturování testů pomocí `describe()` a `it()` :

- Vytvořte vzorovou testovací sadu pro přihlašovací funkci.
- Seskupujte testy do oddělených bloků (např. platné přihlášení, neplatné přihlášení).

2. Implementace hooků pro setup a teardown:

- Napište testy, které využívají `before()` , `beforeEach()` , `after()` a `afterEach()` pro správu stavu testu.
- Příklad aktivity: Zalogujte zprávu nebo nastavte podmínky (např. přejít na login stránku) před každým testem.

3. Tagování a filtrování testů:

- Používejte pojmenovací konvence nebo tagy (např. `@smoke` , `@regression`) v názvech testů.
- Nakonfigurujte a ukažte spouštění testů s pluginem Cypress-grep.

4. Vlastní příkazy:

- Vytvořte vlastní příkaz (např. `cy.login()`) a používejte jej napříč vícero testovacími případy.
- Refaktorujte stávající testovací kód tak, aby tento příkaz využíval.

Zdroje

- **Dokumentace Cypress:**
 - [Cypress Testing Structure](#)
 - [Cypress Commands](#)
- **Příklady testovacích sad:**
 - Hledejte ukázkové projekty na GitHubu, které ukazují dobře organizované Cypress testy.
- **Plugin Cypress-Grep:**
 - [Cypress-Grep GitHub Repository](#)